

LAPORAN PRAKTIKUM
PRAKTIKUM PENAMBANGAN DATA
PERTEMUAN 2
EXPLORATORY DATA ANALYSIS DAN DATA PREPROCESSING



Disusun oleh:

Nama : Hafidz Rizqullah Prasetya
NIM : 24/535493/SV/24243
Kelas : PL5A1
Dosen Pengampu : Dr. Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Isi	2
Tujuan Praktikum	3
Dasar Teori	4
2.1 Artificial Intelligence dan Machine Learning	4
2.1.1 Artificial Intelligence	4
2.1.2 Machine Learning	4
2.2 Exploratory Data Analysis	5
2.3 Data Preprocessing	6
2.3.1 Pembersihan Data dan Imputasi	7
2.3.2 Encoding Atribut Kategorikal	8
2.3.3 Pembagian Dataset dan Scaling	9
Hasil dan Pembahasan	10
3.1 Percobaan Latihan Praktikum	10
3.1.1 Persiapan dan Impor Pustaka	10
3.2 Exploratory Data Analysis	11
3.2.1 Memuat Dataset Bank Churners	11
3.2.2 Pemeriksaan dan Pemisahan Kelas	12
3.2.3 Visualisasi Dataset Bank Churners	12
3.2.4 Visualisasi Dataset IoT	14
3.3 Data Preprocessing	15
3.3.1 Memuat dan Memeriksa Dataset Titanic	15
3.3.2 Distribusi, Korelasi, dan Kualitas Data	16
3.3.3 Imputasi Missing Value	19
3.3.4 Seleksi Fitur dan Encoding	19
3.3.5 Pembagian Dataset dan Scaling	22
3.4 Tugas dan Analisis Praktikum	23
3.4.1 Tugas EDA: Credit Card Customers	23
3.4.1.1 Membaca Dataset	23
3.4.1.2 Tujuan Penggunaan Dataset	24
3.4.1.3 Atribut Input dan Output	24

3.4.1.4	Penjelasan Setiap Atribut	25
3.4.2	Tugas Data Preprocessing: Bengaluru House Price	25
3.4.2.1	Membaca Dataset	25
3.4.2.2	Visualisasi Histogram dan Pola Distribusi	26
3.4.2.3	Korelasi Antar Variabel	27
3.4.2.4	EDA: Descriptive Statistics, Missing Value, dan Duplikasi	27
3.4.2.5	Preprocessing Lanjutan: Encoding, Split, dan Scaling	28
Kesimpulan		30
Daftar Pustaka		31

Tujuan Praktikum

1. Mahasiswa mampu memahami konsep dasar Artificial Intelligence (AI).
2. Mahasiswa mampu mengetahui dasar-dasar Machine Learning (ML).
3. Mahasiswa mampu menerapkan Exploratory Data Analysis (EDA) untuk memahami karakteristik, distribusi, dan kualitas dataset.
4. Mahasiswa mampu menerapkan teknik dasar data preprocessing, meliputi penanganan missing value dan duplikasi, encoding, pembagian dataset, serta scaling fitur.

Dasar Teori

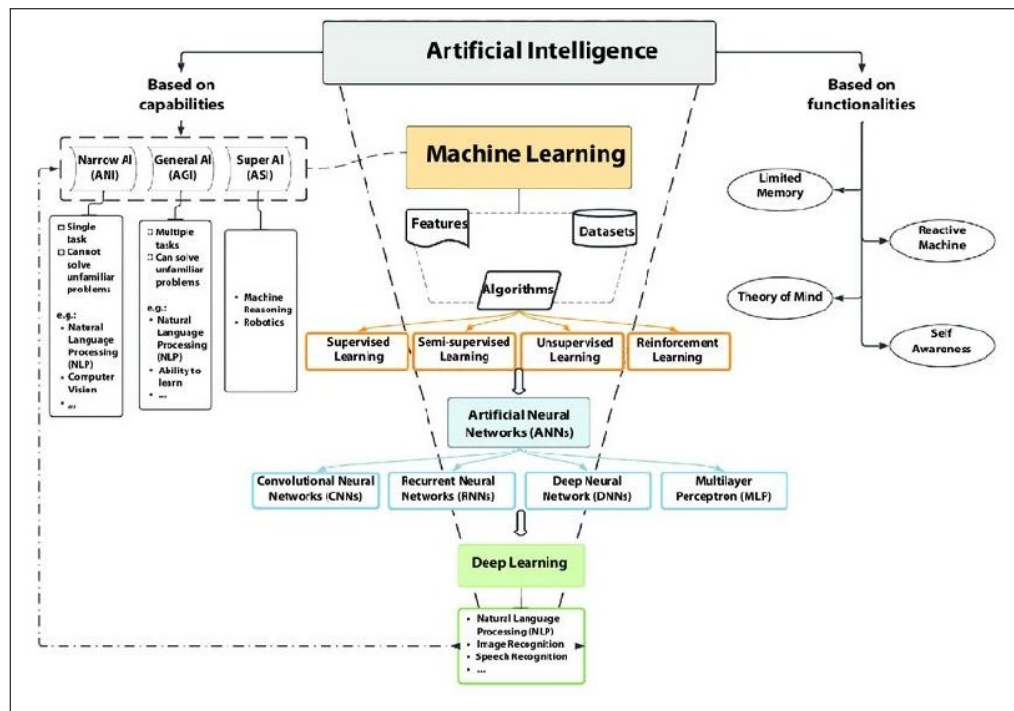
2.1 Artificial Intelligence dan Machine Learning

2.1.1 Artificial Intelligence

Artificial Intelligence (AI) adalah bidang ilmu komputer yang mengembangkan sistem agar mampu melakukan tugas yang umumnya membutuhkan kecerdasan manusia, seperti mengenali pola, belajar dari pengalaman, memahami informasi, dan mengambil keputusan. Dalam penerapannya, kualitas data menentukan kemampuan sistem AI karena data menjadi dasar untuk menemukan pola dan menghasilkan prediksi [1].

2.1.2 Machine Learning

Machine Learning (ML) merupakan cabang AI yang memungkinkan komputer mempelajari pola dari data dan meningkatkan kinerjanya tanpa aturan yang diprogram secara eksplisit untuk setiap kasus. ML dapat dibagi menjadi *supervised learning*, *unsupervised learning*, dan *reinforcement learning*. Pada *supervised learning*, data memiliki label untuk keperluan klasifikasi atau prediksi. Pada *unsupervised learning*, model mencari struktur atau pola tersembunyi tanpa label. Sementara itu, *reinforcement learning* belajar melalui umpan balik berupa penghargaan dan hukuman [5].

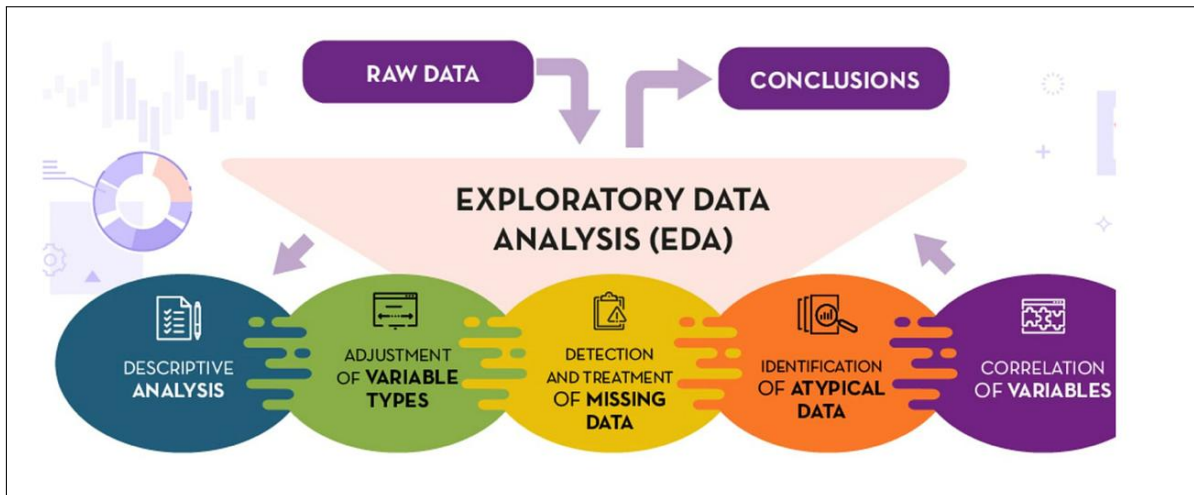


Gambar 1: Klasifikasi Artificial Intelligence, Machine Learning, dan Deep Learning.

2.2 Exploratory Data Analysis

Exploratory Data Analysis (EDA) adalah proses memeriksa dan memahami dataset sebelum dilakukan pemodelan. EDA digunakan untuk mengetahui struktur data, tipe atribut, distribusi nilai, hubungan antarvariabel, outlier, missing value, dan duplikasi. Pemeriksaan dapat dilakukan menggunakan `head()`, `tail()`, `shape`, `describe()`, dan `isnull().sum()` pada `DataFrame` `pandas`.

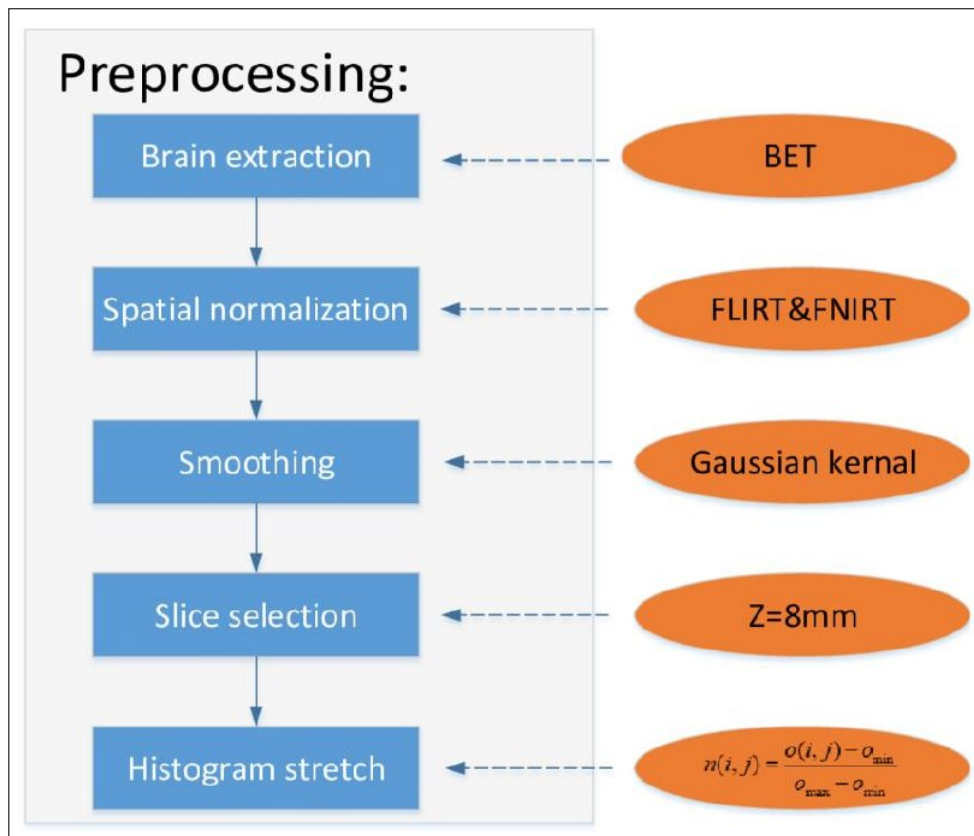
Visualisasi membantu menyampaikan karakteristik data secara lebih mudah. Line plot digunakan untuk melihat perubahan atau tren, pie chart untuk proporsi, bar plot untuk membandingkan kategori, histogram untuk distribusi numerik, scatter plot untuk hubungan dua variabel, box plot untuk ringkasan kuartil dan outlier, serta violin plot untuk bentuk distribusi dan kepadatan data. Korelasi antarvariabel numerik juga dapat dihitung dengan `corr()` untuk mengetahui arah dan kekuatan hubungan linear [2, 6].



Gambar 2: Alur Exploratory Data Analysis mulai dari data mentah hingga menghasilkan kesimpulan.

2.3 Data Preprocessing

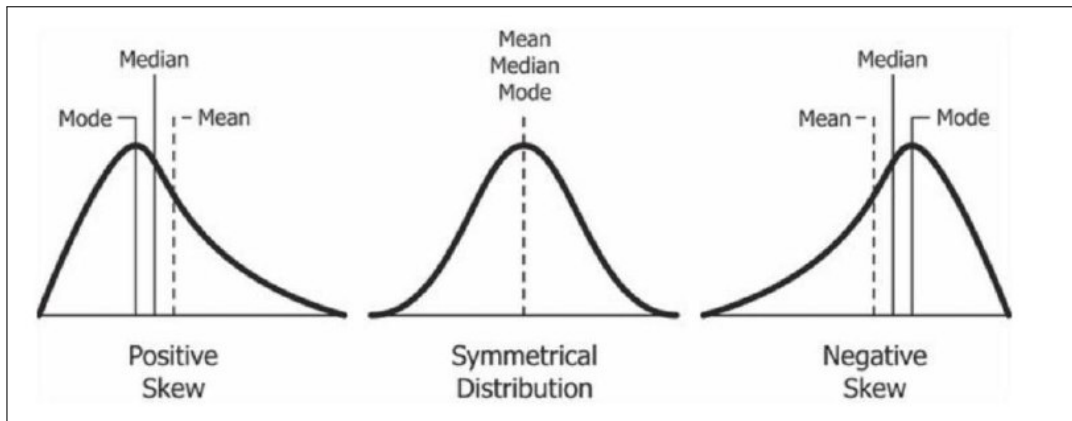
Data preprocessing merupakan serangkaian langkah untuk membersihkan dan mengubah data mentah menjadi data yang siap dianalisis atau digunakan oleh model ML. Tahap ini penting karena data yang tidak konsisten, tidak lengkap, atau memiliki skala berbeda dapat menurunkan kualitas analisis dan performa model.



Gambar 3: Contoh alur data preprocessing yang meliputi ekstraksi, standardisasi spasial, smoothing, pemilihan citra, dan normalisasi intensitas.

2.3.1 Pembersihan Data dan Imputasi

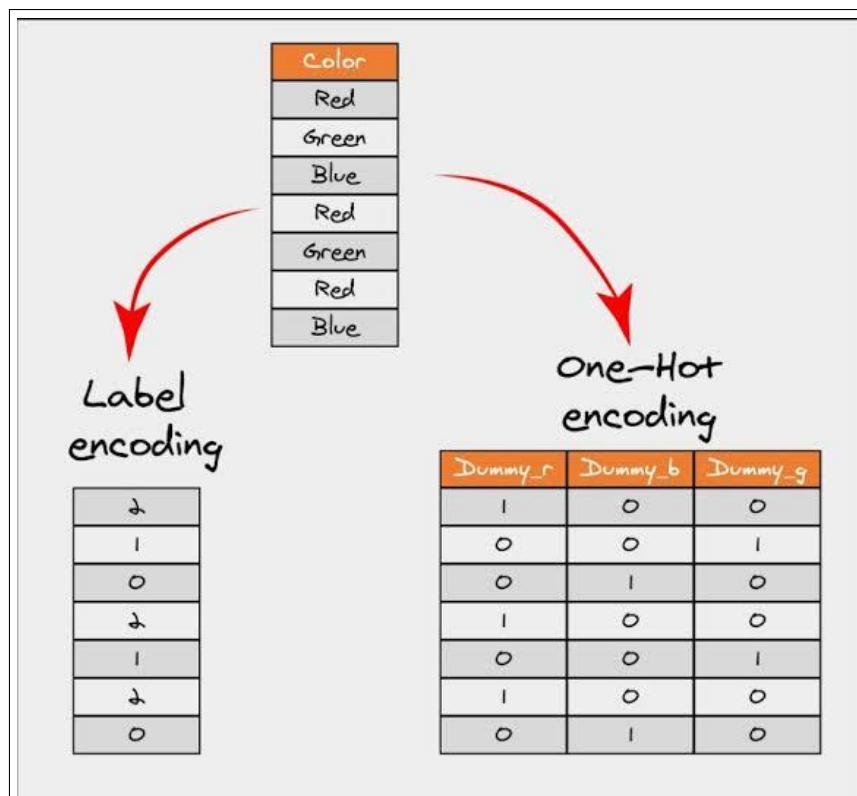
Missing value dapat diperiksa menggunakan `isnull().sum()`, sedangkan baris duplikat dapat diperiksa menggunakan `duplicated().sum()`. Missing value dapat ditangani dengan menghapus baris atau kolom yang bermasalah, atau dengan imputasi. Nilai median sering digunakan untuk atribut numerik karena lebih tahan terhadap outlier, sedangkan modus dapat digunakan untuk atribut kategorikal. Duplikasi perlu dihapus apabila merepresentasikan pengamatan yang tercatat lebih dari satu kali [3].



Gambar 4: Hubungan mean, median, dan mode pada distribusi data sebagai pertimbangan pemilihan metode imputasi.

2.3.2 Encoding Atribut Kategorikal

Algoritma ML umumnya memerlukan input numerik. One-hot encoding mengubah kategori menjadi beberapa kolom biner dan sesuai untuk kategori nominal. Label encoding mengubah kategori menjadi bilangan integer dan perlu digunakan secara hati-hati karena angka dapat dianggap memiliki urutan. Pada fitur kategorikal nominal, one-hot encoding biasanya lebih aman untuk mencegah model menganggap adanya hubungan ordinal yang sebenarnya tidak ada [5, 4].

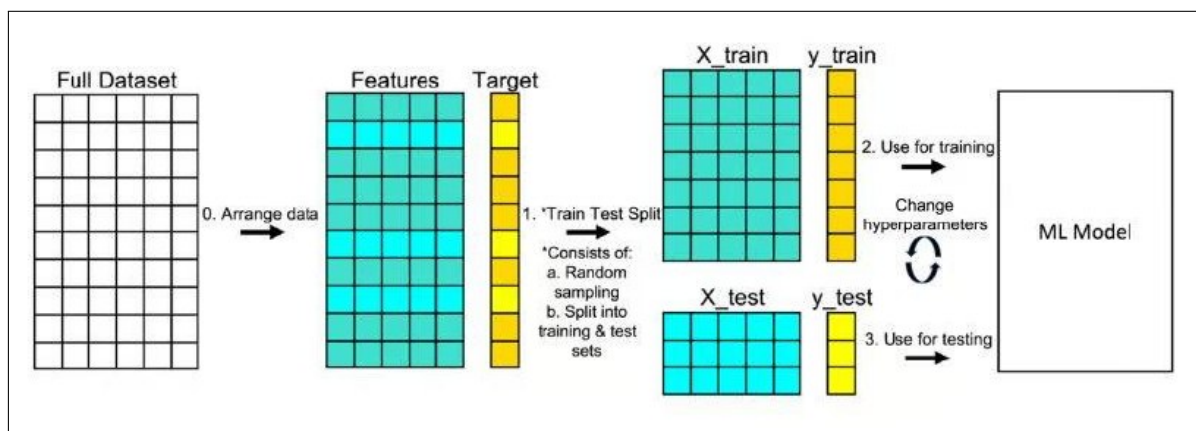


Gambar 5: Perbandingan label encoding dan one-hot encoding pada variabel kategorikal.

2.3.3 Pembagian Dataset dan Scaling

Dataset biasanya dibagi menjadi data latih dan data uji menggunakan `train_test_split()`. Data latih digunakan untuk membangun model, sedangkan data uji digunakan untuk mengukur kemampuan generalisasi model. Parameter pembagian harus disesuaikan dengan instruksi praktikum, misalnya 80:20.

Scaling digunakan agar fitur numerik berada pada skala yang sebanding. *Standardization* menggunakan rata-rata dan simpangan baku sehingga data memiliki rata-rata mendekati nol dan simpangan baku mendekati satu. Sementara itu, *normalization* seperti Min-Max Scaling memetakan nilai ke rentang tertentu, umumnya 0 sampai 1. Parameter scaler harus di-fit pada data latih saja, kemudian digunakan untuk mentransformasi data uji agar tidak terjadi kebocoran informasi [7, 8].



Gambar 6: Alur pembagian features dan target menjadi data latih serta data uji menggunakan train-test split.

Hasil dan Pembahasan

3.1 Percobaan Latihan Praktikum

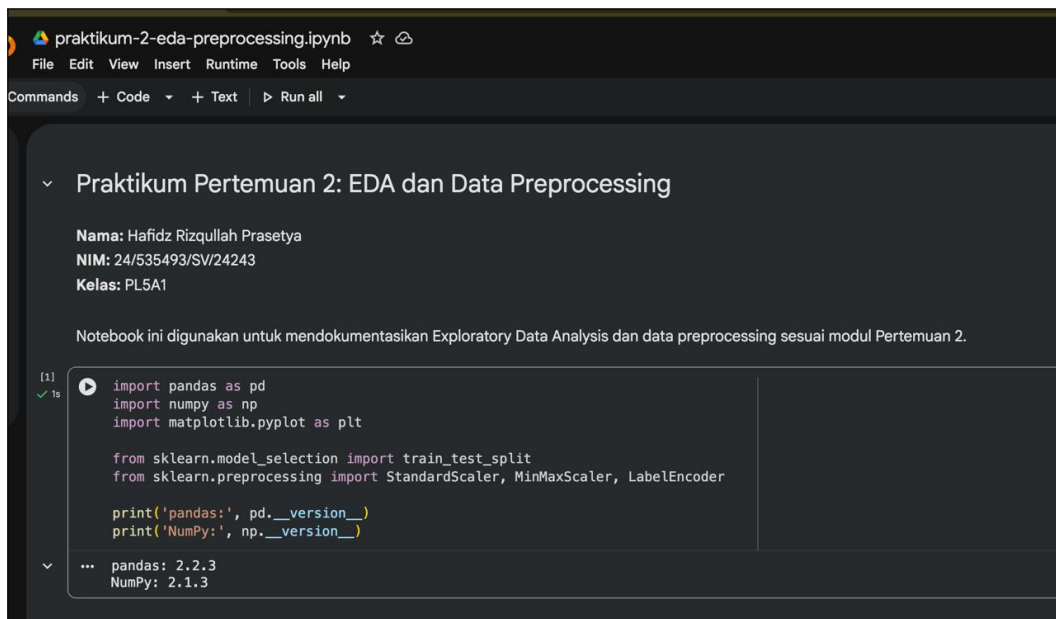
Pada bagian ini saya mendokumentasikan percobaan EDA dan data preprocessing sesuai langkah pada modul Pertemuan 2. Setiap langkah disertai kode yang saya jalankan di Google Colaboratory beserta screenshot hasil eksekusinya. Notebook yang saya gunakan dapat diakses melalui tautan Google Colab berikut: https://colab.research.google.com/drive/1hT4RkcW4MztEHA7YF_YxsZWxfTr3DABz?usp=sharing.

3.1.1 Persiapan dan Impor Pustaka

Saya menyiapkan lingkungan Python dan mengimpor pandas untuk manipulasi data, NumPy untuk komputasi numerik, Matplotlib untuk visualisasi, serta scikit-learn untuk pre-processing.

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder
```

Listing .1: Impor pustaka praktikum.



Gambar 1: Impor pustaka yang digunakan dalam praktikum.

3.2 Exploratory Data Analysis

3.2.1 Memuat Dataset Bank Churners

Untuk latihan EDA, saya menggunakan dataset Bank Churners dari Kaggle. Dataset ini berisi informasi nasabah kartu kredit dengan label Attrition_Flag yang menunjukkan status nasabah. Dataset saya muat menggunakan `pd.read_csv()`.

```
1 df_churning = pd.read_csv('https://raw.githubusercontent.com/ganjar87/
↪ data_science_practice/main/BankChurners.csv')
2 df_churning.head()
```

Listing .2: Memuat dataset Bank Churners.

A. Exploratory Data Analysis

1. Dataset Bank Churners

```
df_churning = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/BankChurners.csv')
display(df_churning.head())
display(df_churning.tail())
print('shape:', df_churning.shape)
```

...

CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_on_book	Months_Inactive_12_mon	Contacts_Count_12_mon	Credit_Limit	Total_Revolving_Bal
0 76860383	Existing Customer	45	M	3	High School	Married	\$60K - \$80K	Blue	39	...	1	3	12691.0
1 81877008	Existing Customer	49	F	5	Graduate	Single	Less than \$40K	Blue	44	...	1	2	8256.0
2 713682108	Existing Customer	51	M	3	Graduate	Married	\$80K - \$120K	Blue	36	...	1	0	3418.0
3 769911858	Existing Customer	40	F	4	High School	Unknown	Less than \$40K	Blue	34	...	4	1	3313.0
4 709106358	Existing Customer	40	M	3	Uneducated	Married	\$60K - \$80K	Blue	21	...	1	0	4716.0

5 rows x 21 columns

CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_on_book	Months_Inactive_12_mon	Contacts_Count_12_mon	Credit_Limit	Total_Revolving_Bal
10132 77296683	Existing Customer	50	M	2	Graduate	Single	\$40K - \$60K	Blue	40	...	2	3	4003.0
10133 71063823	Attrited Customer	41	M	2	Unknown	Divorced	\$40K - \$60K	Blue	25	...	2	3	4277.0
10134 716506083	Attrited Customer	44	F	1	High School	Married	Less than \$40K	Blue	38	...	3	4	5409.0
10125 717408983	Attrited Customer	30	M	2	Graduate	Unknown	\$40K - \$60K	Blue	35	...	3	0	5281.0
10126 714337233	Attrited Customer	43	F	2	Graduate	Married	Less than \$40K	Silver	25	...	2	4	10388.0

5 rows x 21 columns
shape: (18127, 21)

Gambar 2: Lima baris pertama dan terakhir dataset Bank Churners.

3.2.2 Pemeriksaan dan Pemisahan Kelas

Saya menggunakan `tail()` untuk memeriksa baris terakhir dataset. Setelah itu, saya memisahkan data menjadi nasabah aktif dan nasabah yang berhenti berdasarkan kolom `Attrition_Flag`.

```
1 df_churning.tail()
2 existing_data = df_churning[df_churning['Attrition_Flag'] == 'Existing Customer']
3 attrited_data = df_churning[df_churning['Attrition_Flag'] == 'Attrited Customer']
```

Listing .3: Pemeriksaan akhir dan pemisahan kelas.

```
2. Pemisahan Kelas

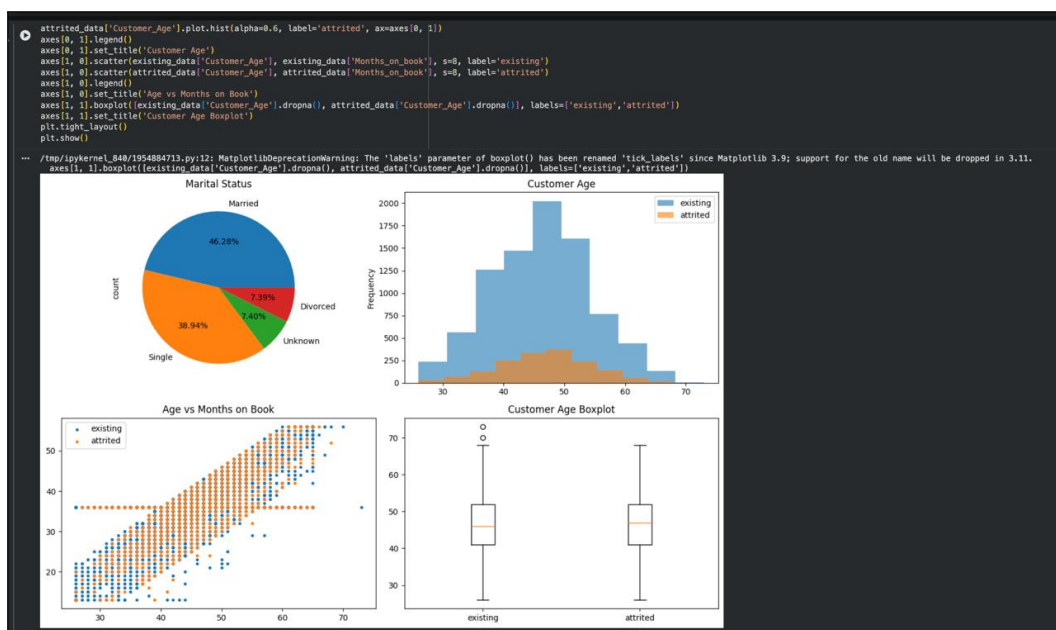
existing_data = df_churning[df_churning['Attrition_Flag'] == 'Existing Customer']
attrited_data = df_churning[df_churning['Attrition_Flag'] == 'Attrited Customer']
print('Existing:', len(existing_data))
print('Attrited:', len(attrited_data))

... Existing: 8500
    Attrited: 1627
```

Gambar 3: Pemeriksaan baris terakhir dan pemisahan kelas nasabah.

3.2.3 Visualisasi Dataset Bank Churners

Saya membuat line chart, pie chart, bar plot, dan stacked bar plot untuk melihat tren serta perbandingan kategori. Untuk data numerik, saya menggunakan histogram, scatter plot, box plot, dan violin plot. Visualisasi tersebut membantu saya mengamati distribusi usia, perbandingan status nasabah, serta kemungkinan outlier.



Gambar 4: Visualisasi EDA dataset Bank Churners.

Hasil keseluruhan visualisasi tersebut ditampilkan pada Gambar 4. Selain itu, modul juga memperkenalkan beberapa teknik visualisasi pelengkap yang saya pelajari, yaitu line chart dengan data dummy, subplot, annotation, axis, dan legend.

Line Chart dengan Data Dummy. Sebelum menerapkan line chart pada data nyata, saya mempelajarinya menggunakan data dummy agar memahami cara menampilkan dua seri data dalam satu grafik dengan marker yang berbeda.

```
1 x = [1, 2, 3, 4, 5]
2 y1 = [2, 3, 5, 7, 11]
3 y2 = [1, 4, 9, 16, 25]
4
5 plt.plot(x, y1, marker='o')
6 plt.plot(x, y2, marker='X')
7 plt.show()
```

Listing .4: Line chart dengan data dummy.

Subplot. Teknik subplot digunakan untuk menampilkan beberapa grafik sekaligus dalam satu figure. Saya membuat grid 2×2 yang berisi box plot dan histogram untuk masing-masing kelompok nasabah agar perbandingan distribusi usia lebih mudah diamati.

```
1 plt.subplot(2, 2, 1)
2 plt.boxplot(existing_data['Customer_Age'])
3 plt.title('Box Plot Existing')
4
5 plt.subplot(2, 2, 2)
6 plt.boxplot(attrited_data['Customer_Age'])
7 plt.title('Box Plot Attrited')
8
9 plt.subplot(2, 2, 3)
10 plt.hist(existing_data['Customer_Age'])
11 plt.title('Histogram Existing')
12
13 plt.subplot(2, 2, 4)
14 plt.hist(attrited_data['Customer_Age'])
15 plt.title('Histogram Attrited')
16
17 plt.show()
```

Listing .5: Subplot box plot dan histogram.

Annotation. Annotation digunakan untuk menambahkan informasi statistik langsung pada grafik menggunakan `plt.text()`. Saya menambahkan nilai median dan mean usia pada histogram `Customer_Age`.

```
1 plt.hist(df_churning['Customer_Age'])
2 median_age = df_churning['Customer_Age'].median()
3 mean_age = df_churning['Customer_Age'].mean()
4 plt.text(median_age, 900, f'Median Age: {median_age:.2f}')
```

```

5 plt.text(mean_age, 850, f'Mean Age: {mean_age:.2f}')
6 plt.show()

```

Listing .6: Annotation median dan mean pada histogram.

Axis. Pengaturan axis dilakukan untuk mengendalikan rentang nilai sumbu dan memberi label agar grafik lebih mudah dibaca. Saya mengatur label sumbu serta batas sumbu-y menggunakan `plt.ylim()`.

```

1 hari = ['Senin', 'Selasa', 'Rabu', 'Kamis', 'Jumat']
2 suhu1 = [25, 26, 24, 27, 28]
3 suhu2 = [22, 23, 21, 24, 25]
4
5 plt.plot(hari, suhu1, marker='o', label='Hari 1')
6 plt.plot(hari, suhu2, marker='X', label='Hari 2')
7 plt.legend()
8 plt.ylabel('Suhu')
9 plt.xlabel('Hari')
10 plt.ylim([-20, 30])
11 plt.show()

```

Listing .7: Pengaturan axis pada line chart.

Legend. Legend digunakan untuk membedakan kelompok data dalam satu grafik. Saya menambahkan legend pada histogram yang menampilkan dua kelompok nasabah.

```

1 plt.hist(existing_data['Customer_Age'], label='existing')
2 plt.hist(attrited_data['Customer_Age'], label='attrited')
3 plt.legend()
4 plt.show()

```

Listing .8: Legend pada histogram.

3.2.4 Visualisasi Dataset IoT

Saya memuat dataset IoT dan memvisualisasikan perubahan kelembapan terhadap waktu menggunakan line chart.

```

1 df_iot = pd.read_csv('https://raw.githubusercontent.com/ganjar87/
↳ data_science_practice/main/datatraining.txt')
2 plt.figure(figsize=(15, 3))
3 plt.plot(df_iot['date'], df_iot['Humidity'])
4 plt.xlabel('Date')
5 plt.ylabel('Humidity')
6 plt.show()

```

Listing .9: Visualisasi kelembapan dataset IoT.



Gambar 5: Perubahan kelembapan terhadap waktu pada dataset IoT.

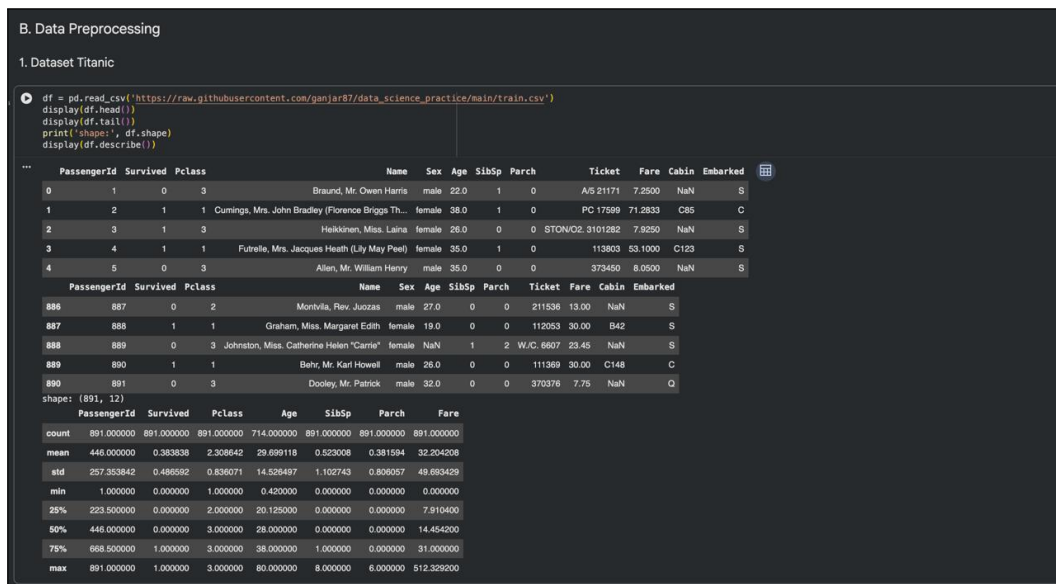
3.3 Data Preprocessing

3.3.1 Memuat dan Memeriksa Dataset Titanic

Untuk latihan preprocessing, saya menggunakan dataset Titanic. Dataset ini memiliki target Survived. Saya memeriksa data awal, ukuran dataset, statistik deskriptif, dan distribusi target sebelum melakukan transformasi.

```
1 df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/
↪ main/train.csv')
2 df.head()
3 df.tail()
4 df.shape
5 df.describe()
```

Listing .10: Pemeriksaan awal dataset Titanic.



Gambar 6: Pemeriksaan awal dataset Titanic.

3.3.2 Distribusi, Korelasi, dan Kualitas Data

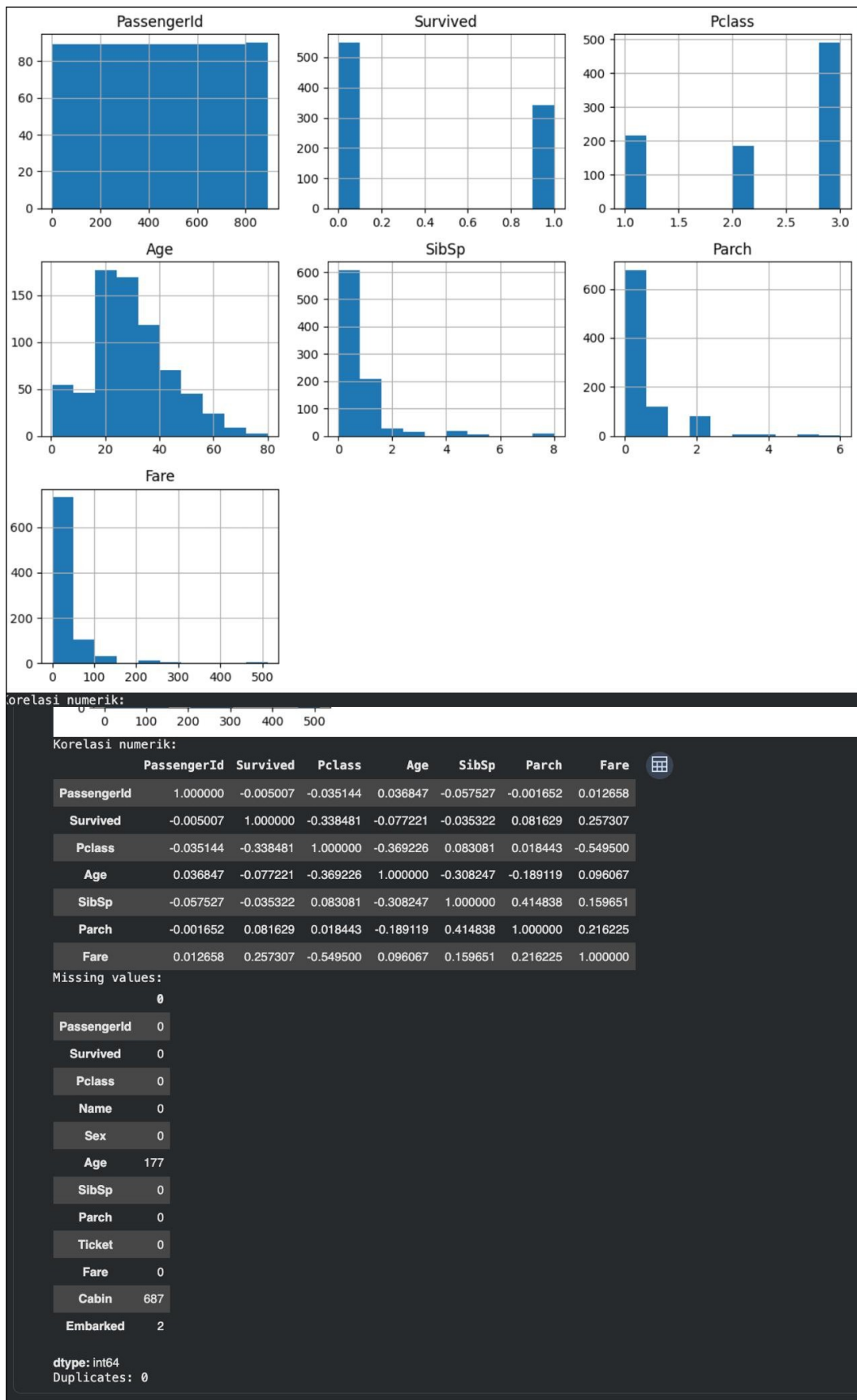
Saya membuat pie chart target dan histogram fitur numerik. Saya menghitung korelasi menggunakan `corr(numeric_only=True)` serta memeriksa missing value dan duplikasi. Hasil pemeriksaan ini menjadi dasar keputusan pembersihan data.

```
1 df['Survived'].value_counts().plot.pie(autopct='%0.2f%%')
2 plt.show()
3 df.hist(figsize=(10, 8))
4 plt.show()
5 df.corr(numeric_only=True)
6 df.isnull().sum()
7 df.duplicated().sum()
```

Listing .11: Pemeriksaan distribusi dan kualitas data.



Gambar 7: Distribusi target Survived pada dataset Titanic.



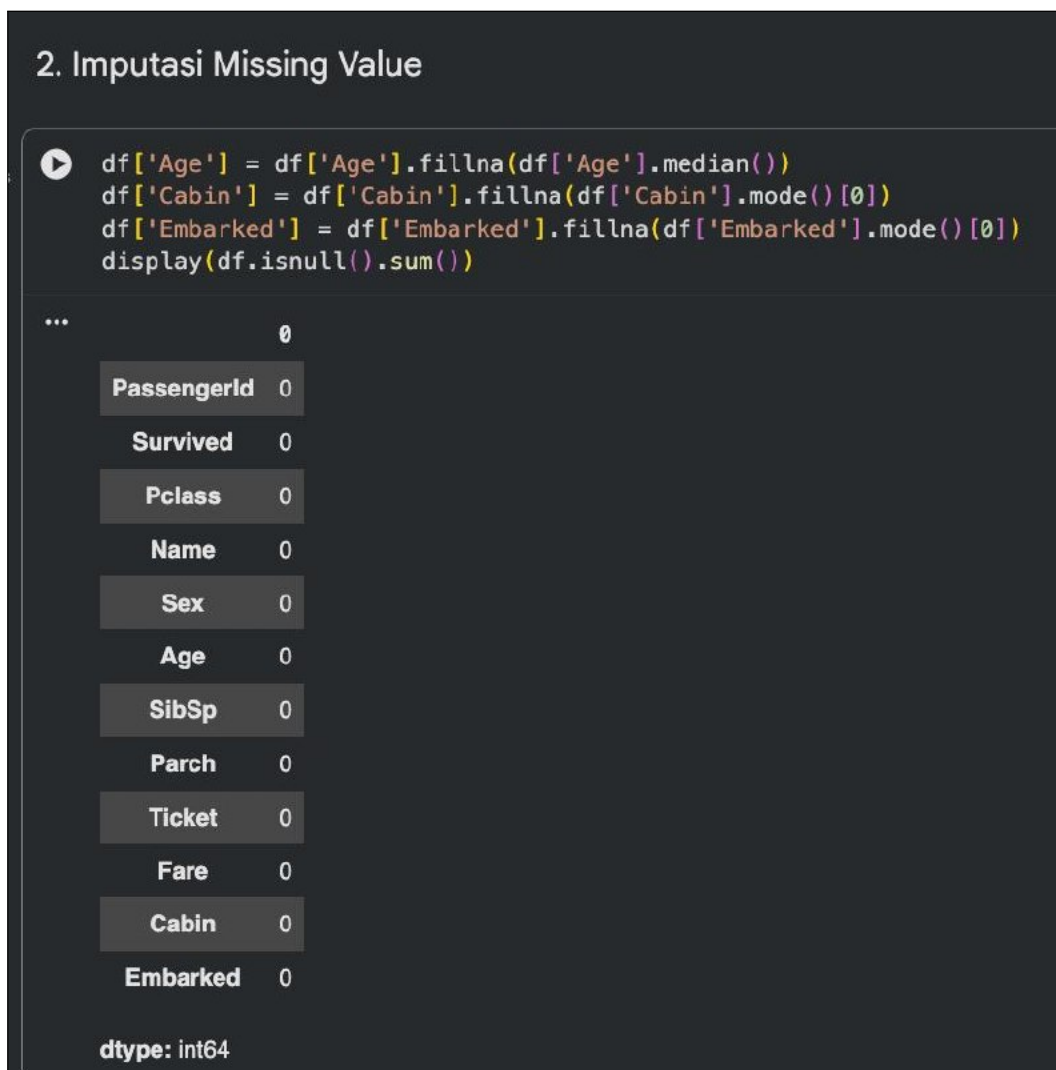
Gambar 8: Histogram fitur numerik, korelasi, missing value, dan duplikasi dataset Titanic.

3.3.3 Imputasi Missing Value

Saya mengisi missing value pada Age menggunakan median karena median lebih tahan terhadap outlier. Kolom kategorikal Cabin dan Embarked saya isi menggunakan modus. Setelah itu, saya memeriksa kembali missing value.

```
1 df['Age'] = df['Age'].fillna(df['Age'].median())
2 df['Cabin'] = df['Cabin'].fillna(df['Cabin'].mode()[0])
3 df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
4 df.isnull().sum()
```

Listing .12: Imputasi missing value.



The screenshot shows a Jupyter Notebook interface. The top section is titled "2. Imputasi Missing Value". Below the title, there is a code cell with the following Python code:

```
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Cabin'] = df['Cabin'].fillna(df['Cabin'].mode()[0])
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
display(df.isnull().sum())
```

Below the code cell, the output of the code is displayed. It shows a summary of missing values for each column in the DataFrame. The output is as follows:

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	0
Embarked	0

At the bottom of the output, it says "dtype: int64".

Gambar 9: Pengecekan missing value setelah imputasi.

3.3.4 Seleksi Fitur dan Encoding

Saya memisahkan fitur dan target dengan menghapus kolom identitas atau kolom yang tidak digunakan. Atribut kategorikal Sex dan Embarked kemudian saya ubah menjadi

numerik menggunakan one-hot encoding. Saya juga mempelajari label encoding sebagai metode alternatif.

```
1 X = df.drop(['PassengerId', 'Name', 'Survived', 'Cabin', 'Ticket'], axis=1)
2 y = df['Survived']
3 X = pd.get_dummies(X, columns=['Sex', 'Embarked'], drop_first=True)
4 X = X.astype(int)
5 display(X.head())
```

Listing .13: Seleksi fitur dan one-hot encoding.

3. Seleksi Fitur dan Encoding

```
X = df.drop(['PassengerId', 'Name', 'Survived', 'Cabin', 'Ticket'], axis=1)
y = df['Survived']
X = pd.get_dummies(X, columns=['Sex', 'Embarked'], drop_first=True)
display(X.head())
```

...	Pclass	Age	SibSp	Parch	Fare	Sex_male	Embarked_Q	Embarked_S
0	3	22.0	1	0	7.2500	True	False	True
1	1	38.0	1	0	71.2833	False	False	False
2	3	26.0	0	0	7.9250	False	False	True
3	1	35.0	1	0	53.1000	False	False	True
4	3	35.0	0	0	8.0500	True	False	True

Gambar 10: Hasil encoding atribut kategorikal.

Selain `pd.get_dummies()`, modul juga memperkenalkan penerapan one-hot encoding menggunakan `OneHotEncoder` dari `scikit-learn`. Encoder di-fit pada kolom kategorikal, lalu hasilnya diubah menjadi `DataFrame` biner.

```
1 from sklearn.preprocessing import OneHotEncoder
2
3 encoder = OneHotEncoder(drop='first')
4 encoded_data = encoder.fit_transform(df[['Sex', 'Embarked']])
5 df_ohe = pd.DataFrame(encoded_data.toarray(), columns=encoder.get_feature_names_out
6 ↪(['Sex', 'Embarked']))
7 display(df_ohe.head())
```

Listing .14: One-hot encoding dengan `OneHotEncoder`.

3a. One-Hot Encoding dengan OneHotEncoder

Variasi one-hot encoding menggunakan OneHotEncoder dari scikit-learn.

```

1 from sklearn.preprocessing import OneHotEncoder
2
3 encoder = OneHotEncoder(drop='first')
4 encoded_data = encoder.fit_transform(df[['Sex', 'Embarked']])
5 df_ohe = pd.DataFrame(encoded_data.toarray(), columns=encoder.get_feature_names_out(['Sex', 'Embarked']))
6 display(df_ohe.head())

```

	Sex_male	Embarked_Q	Embarked_S
0	1.0	0.0	1.0
1	0.0	0.0	0.0
2	0.0	0.0	1.0
3	0.0	0.0	1.0
4	1.0	0.0	1.0

Gambar 11: Hasil one-hot encoding menggunakan OneHotEncoder.

Hasilnya berupa kolom-kolom biner seperti Sex_male, Embarked_Q, dan Embarked_S yang siap digunakan sebagai fitur model.

Sebagai metode alternatif, saya juga mempelajari label encoding menggunakan LabelEncoder. Metode ini mengubah setiap kategori menjadi bilangan integer sehingga lebih efisien untuk kolom dengan banyak kategori, tetapi perlu digunakan secara hati-hati karena bilangan yang dihasilkan dapat dianggap memiliki urutan oleh model.

```

1 X_label = df.drop(['PassengerId', 'Name', 'Survived', 'Cabin', 'Ticket'], axis=1)
2 for col in ['Sex', 'Embarked']:
3     X_label[col] = LabelEncoder().fit_transform(X_label[col])
4 y_encoded = LabelEncoder().fit_transform(y)
5 display(X_label.head())

```

Listing .15: Label encoding untuk fitur dan target.

3b. Label Encoding

Metode alternatif: mengubah kategori menjadi bilangan integer.

```

[15]
✓ 0s
1 X_label = df.drop(['PassengerId', 'Name', 'Survived', 'Cabin', 'Ticket'], axis=1)
2 for col in ['Sex', 'Embarked']:
3     X_label[col] = LabelEncoder().fit_transform(X_label[col])
4 y_encoded = LabelEncoder().fit_transform(y)
5 display(X_label.head())

```

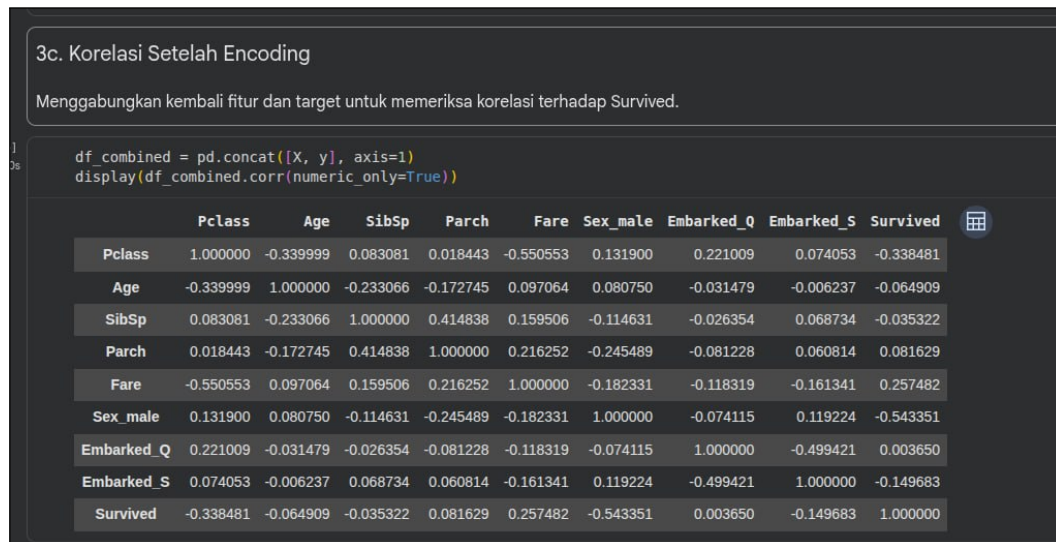
	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	1	22.0	1	0	7.2500	2
1	1	0	38.0	1	0	71.2833	0
2	3	0	26.0	0	0	7.9250	2
3	1	0	35.0	1	0	53.1000	2
4	3	1	35.0	0	0	8.0500	2

Gambar 12: Hasil label encoding pada fitur dan target.

Setelah encoding, saya menggabungkan kembali fitur dan target untuk memeriksa korelasi setiap atribut terhadap Survived.

```
1 df_combined = pd.concat([X, y], axis=1)
2 display(df_combined.corr(numeric_only=True))
```

Listing .16: Korelasi atribut terhadap target setelah encoding.



Gambar 13: Matriks korelasi atribut terhadap target setelah encoding.

Dari matriks korelasi tersebut, terlihat bahwa Sex_male memiliki korelasi negatif kuat dengan Survived (-0,54), artinya penumpang laki-laki cenderung lebih kecil kemungkinannya untuk selamat. Pclass berkorelasi negatif (-0,34), sedangkan Fare berkorelasi positif (0,26) terhadap Survived. Hasil ini konsisten dengan analisis korelasi sebelum encoding.

3.3.5 Pembagian Dataset dan Scaling

Saya membagi dataset dengan proporsi 70:30 sesuai modul (`test_size=0.3`). Data latih digunakan untuk proses pembelajaran, sedangkan data uji digunakan untuk evaluasi. Saya menerapkan standardization dengan `StandardScaler` dan normalization dengan `MinMaxScaler`. Scaler di-fit hanya pada data latih untuk mencegah data leakage.

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y, test_size=0.3, random_state=42
3 )
4 print('X_train:', X_train.shape, 'X_test:', X_test.shape)
5 standard_scaler = StandardScaler()
6 X_train_standard = standard_scaler.fit_transform(X_train)
7 X_test_standard = standard_scaler.transform(X_test)
8 minmax_scaler = MinMaxScaler()
9 X_train_normalized = minmax_scaler.fit_transform(X_train)
10 X_test_normalized = minmax_scaler.transform(X_test)
```

```

11 print('standard mean:', X_train_standard.mean(axis=0)[:3])
12 print('normalized min/max:', X_train_normalized.min(), X_train_normalized.max())

```

Listing .17: Train-test split dan scaling.

```

4. Train-Test Split dan Scaling

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
print('X_train:', X_train.shape, 'X_test:', X_test.shape)
standard_scaler = StandardScaler()
X_train_standard = standard_scaler.fit_transform(X_train)
X_test_standard = standard_scaler.transform(X_test)
minmax_scaler = MinMaxScaler()
X_train_normalized = minmax_scaler.fit_transform(X_train)
X_test_normalized = minmax_scaler.transform(X_test)
print('standard mean:', X_train_standard.mean(axis=0)[:3])
print('normalized min/max:', X_train_normalized.min(), X_train_normalized.max())

... X_train: (623, 8) X_test: (268, 8)
standard mean: [ 2.53765263e-16  5.41746067e-17 -3.99181312e-17]
normalized min/max: 0.0 1.0

```

Gambar 14: Pembagian dataset dan scaling fitur.

3.4 Tugas dan Analisis Praktikum

3.4.1 Tugas EDA: Credit Card Customers

3.4.1.1 Membaca Dataset

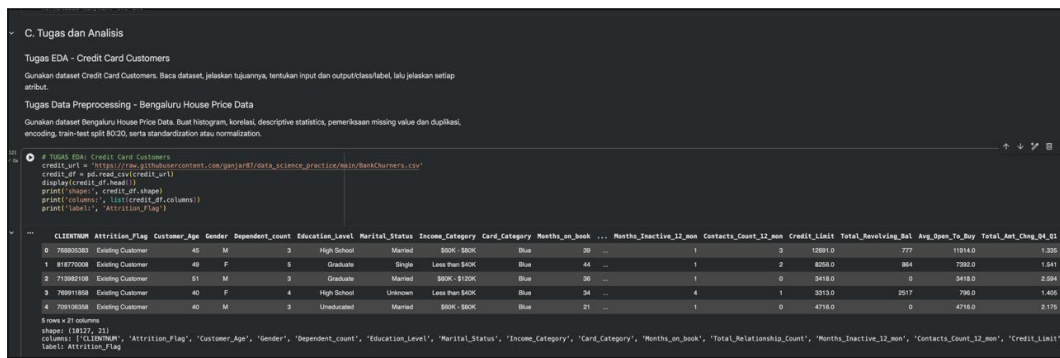
Dataset *Credit Card Customers* saya muat menggunakan `pd.read_csv()` dari sumber data yang tersedia. Dataset ini memiliki 10.127 baris dan 21 kolom.

```

1 credit_url = 'https://raw.githubusercontent.com/ganjar87/data_science_practice/main/
  ↳ BankChurners.csv'
2 credit_df = pd.read_csv(credit_url)
3 display(credit_df.head())
4 print('shape:', credit_df.shape)
5 print('columns:', list(credit_df.columns))
6 print('label:', 'Attrition_Flag')

```

Listing .18: Memuat dataset Credit Card Customers.



Gambar 15: Lima baris pertama dataset Credit Card Customers.

3.4.1.2 Tujuan Penggunaan Dataset

Dataset *Credit Card Customers* digunakan untuk menganalisis perilaku nasabah kartu kredit, khususnya untuk memprediksi apakah seorang nasabah akan tetap menggunakan kartu kreditnya (*Existing Customer*) atau berhenti berlangganan (*Attrited Customer*). Analisis ini penting bagi perusahaan kartu kredit untuk memahami faktor-faktor yang memengaruhi churn nasabah, sehingga dapat mengambil langkah retensi yang tepat.

3.4.1.3 Atribut Input dan Output

- **Atribut input (fitur):** Seluruh kolom selain *Attrition_Flag*, meliputi *CLIENTNUM*, *Customer_Age*, *Gender*, *Dependent_count*, *Education_Level*, *Marital_Status*, *Income_Category*, *Card_Category*, *Months_on_book*, *Total_Relationship_Count*, *Months_Inactive_12_mon*, *Contacts_Count_12_mon*, *Credit_Limit*, *Total_Revolving_Bal*, *Avg_Open_To_Buy*, *Total_Amt_Chng_Q4_Q1*, *Total_Trans_Amt*, *Total_Trans_Ct*, *Total_Ct_Chng_Q4_Q1*, dan *Avg_Utilization_Ratio*.
- **Atribut output (label/target):** *Attrition_Flag* menunjukkan status nasabah (*Existing Customer* atau *Attrited Customer*).

3.4.1.4 Penjelasan Setiap Atribut

Atribut	Penjelasan
CLIENTNUM	Nomor identifikasi unik nasabah.
Attrition_Flag	Status nasabah: Existing atau Attrited (target).
Customer_Age	Usia nasabah dalam tahun.
Gender	Jenis kelamin nasabah (M/F).
Dependent_count	Jumlah tanggungan nasabah.
Education_Level	Tingkat pendidikan terakhir nasabah.
Marital_Status	Status pernikahan nasabah.
Income_Category	Kategori pendapatan tahunan nasabah.
Card_Category	Kategori jenis kartu kredit.
Months_on_book	Lama menjadi nasabah dalam bulan.
Total_Relationship_Count	Jumlah produk yang dimiliki nasabah.
Months_Inactive_12_mon	Jumlah bulan tidak aktif dalam 12 bulan terakhir.
Contacts_Count_12_mon	Jumlah kontak dalam 12 bulan terakhir.
Credit_Limit	Batas kredit kartu.
Total_Revolving_Bal	Saldo revolving (hutang berjalan).
Avg_Open_To_Buy	Rata-rata ketersediaan kredit.
Total_Amt_Chng_Q4_Q1	Perubahan jumlah transaksi Q4 ke Q1.
Total_Trans_Amt	Total jumlah transaksi dalam 12 bulan.
Total_Trans_Ct	Total frekuensi transaksi dalam 12 bulan.
Total_Ct_Chng_Q4_Q1	Perubahan frekuensi transaksi Q4 ke Q1.
Avg_Utilization_Ratio	Rasio rata-rata penggunaan kartu.

Table .1: Deskripsi atribut dataset Credit Card Customers.

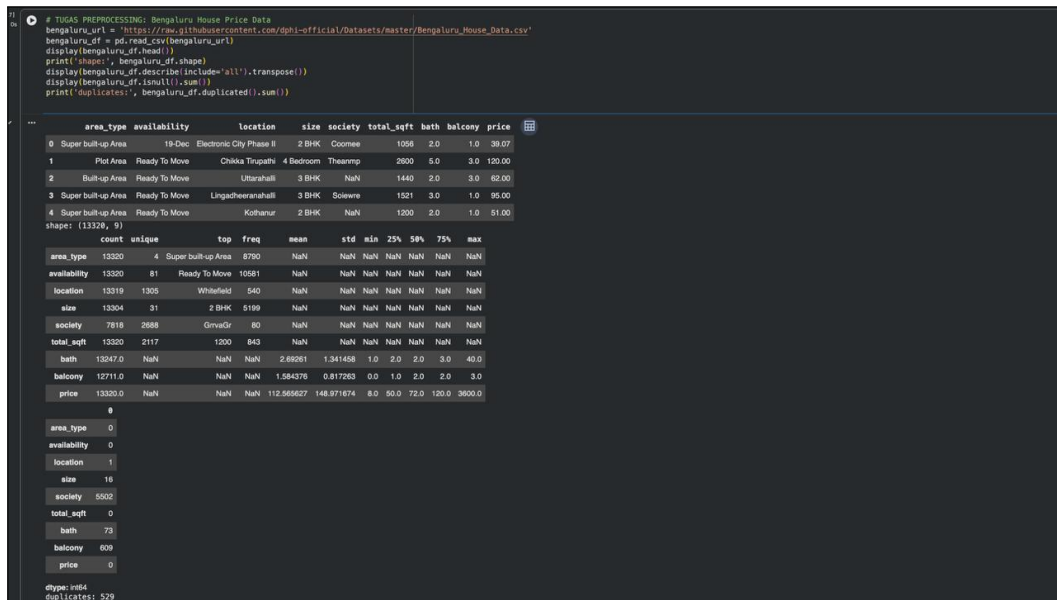
3.4.2 Tugas Data Preprocessing: Bengaluru House Price

3.4.2.1 Membaca Dataset

Dataset *Bengaluru House Price Data* saya muat menggunakan `pd.read_csv()`. Dataset ini berisi 13.320 baris dan 9 kolom yang mencakup informasi properti di Bengaluru, India.

```
1 bengaluru_url = 'https://raw.githubusercontent.com/dphi-official/Datasets/master/  
↪ Bengaluru_House_Data.csv'  
2 bengaluru_df = pd.read_csv(bengaluru_url)  
3 display(bengaluru_df.head())  
4 print('shape:', bengaluru_df.shape)  
5 display(bengaluru_df.describe(include='all').transpose())  
6 display(bengaluru_df.isnull().sum())  
7 print('duplicates:', bengaluru_df.duplicated().sum())
```

Listing .19: Memuat dataset Bengaluru House Price.



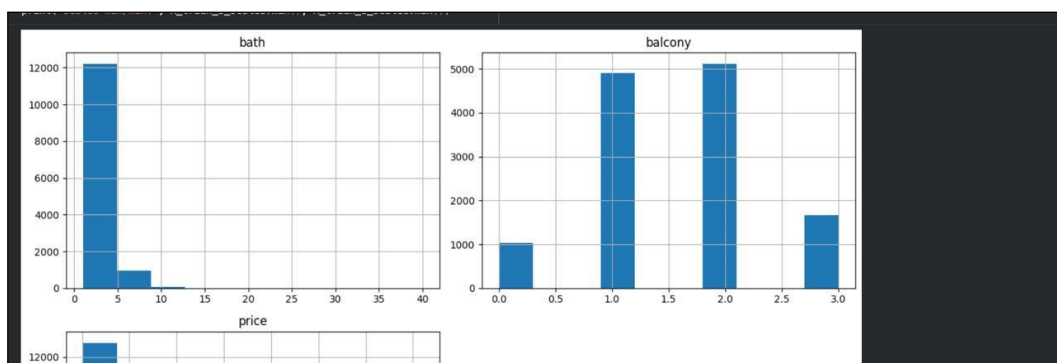
Gambar 16: Pemeriksaan awal dataset Bengaluru House Price.

3.4.2.2 Visualisasi Histogram dan Pola Distribusi

Saya membuat histogram untuk variabel numerik bath, balcony, dan price guna mengamati pola distribusinya.

```
1 numeric_bengaluru = bengaluru_df.select_dtypes(include='number')
2 numeric_bengaluru.hist(figsize=(12, 8))
3 plt.tight_layout()
4 plt.show()
```

Listing .20: Histogram variabel numerik Bengaluru.



Gambar 17: Histogram variabel numerik bath, balcony, dan price dataset Bengaluru.

Berdasarkan histogram yang dihasilkan, terlihat bahwa:

- bath: Distribusi *right-skewed*, mayoritas properti memiliki 1–5 kamar mandi dengan puncak di sekitar 2 kamar mandi, namun terdapat outlier hingga 40 kamar mandi.

- **balcony**: Distribusi cenderung *right-skewed*, sebagian besar properti memiliki 0–2 balkon (dengan puncak di 1 dan 2 balkon), dan sedikit yang memiliki 3 balkon atau lebih.
- **price**: Distribusi *right-skewed* dengan *long tail*, mayoritas properti berada di rentang harga rendah (di bawah 500), namun terdapat outlier dengan harga hingga 3.600.

3.4.2.3 Korelasi Antar Variabel

Saya menghitung korelasi antar variabel numerik untuk melihat hubungan linear di antara mereka.

```
display(numeric_bengaluru.corr())
```

Listing .21: Korelasi variabel numerik Bengaluru.



Gambar 18: Matriks korelasi variabel numerik dataset Bengaluru.

Hasil korelasi menunjukkan:

- **bath** dan **price** memiliki korelasi positif sedang (0.456), artinya semakin banyak kamar mandi, harga properti cenderung semakin tinggi.
- **balcony** dan **price** memiliki korelasi positif lemah (0.120), artinya jumlah balkon tidak terlalu memengaruhi harga.
- **bath** dan **balcony** memiliki korelasi positif lemah (0.204), menunjukkan keduanya relatif independen.

3.4.2.4 EDA: Descriptive Statistics, Missing Value, dan Duplikasi

Saya melakukan pemeriksaan kualitas data menggunakan `describe()`, `isnull().sum()`, dan `duplicated().sum()`.

```
1 display(bengaluru_df.describe(include='all').transpose())
2 display(bengaluru_df.isnull().sum())
3 print('duplicates:', bengaluru_df.duplicated().sum())
```

Listing .22: Pemeriksaan kualitas data Bengaluru.

Hasil pemeriksaan menunjukkan:

- **Missing value:** Kolom society memiliki 5.502 missing value, balcony 609, bath 73, size 16, dan location 1. Kolom society perlu ditangani karena lebih dari 40% data hilang.
- **Duplikasi:** Ditemukan 529 baris duplikat yang perlu dihapus agar tidak memengaruhi analisis.
- **Tindakan:** Baris duplikat dihapus dengan `drop_duplicates()`. Missing value pada kolom numerik diisi dengan median, sedangkan kolom kategorikal diisi dengan modus. Kolom society dapat dihapus karena terlalu banyak missing value.

3.4.2.5 Preprocessing Lanjutan: Encoding, Split, dan Scaling

```

1 # Hapus duplikat
2 bengaluru_model_df = bengaluru_df.drop_duplicates().copy()
3
4 # Ekstrak ukuran BHK dari kolom size
5 bengaluru_model_df['size_bhk'] = bengaluru_model_df['size'].str.extract(r'(\d+)').
  ↳ astype(float)
6
7 # Isi missing value
8 bengaluru_model_df['size_bhk'] = bengaluru_model_df['size_bhk'].fillna(
  ↳ bengaluru_model_df['size_bhk'].median())
9 bengaluru_model_df['bath'] = bengaluru_model_df['bath'].fillna(bengaluru_model_df['
  ↳ bath'].median())
10 bengaluru_model_df['balcony'] = bengaluru_model_df['balcony'].fillna(
  ↳ bengaluru_model_df['balcony'].median())
11
12 # One-hot encoding untuk area_type dan availability
13 bengaluru_model_df = pd.get_dummies(bengaluru_model_df, columns=['area_type', '
  ↳ availability'], drop_first=True)
14
15 # Label encoding untuk location (banyak kategori unik)
16 le = LabelEncoder()
17 bengaluru_model_df['location_encoded'] = le.fit_transform(bengaluru_model_df['
  ↳ location'].fillna('Unknown'))
18
19 # Hapus kolom yang tidak digunakan
20 bengaluru_model_df = bengaluru_model_df.drop(['size', 'location', 'society', '
  ↳ total_sqft'], axis=1)
21
22 # Pisahkan fitur dan target
23 X_bengaluru = bengaluru_model_df.drop('price', axis=1)
24 y_bengaluru = bengaluru_model_df['price']
25
26 # Train-test split 80:20
27 X_train_b, X_test_b, y_train_b, y_test_b = train_test_split(

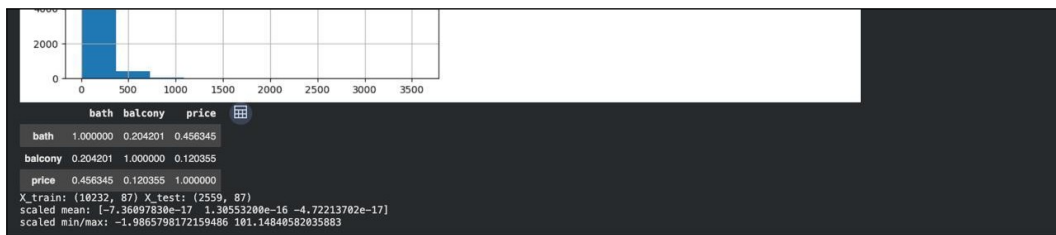
```

```

28 X_bengaluru, y_bengaluru, test_size=0.2, random_state=42
29 )
30
31 # Standardization
32 scaler_b = StandardScaler()
33 X_train_b_scaled = scaler_b.fit_transform(X_train_b)
34 X_test_b_scaled = scaler_b.transform(X_test_b)
35
36 print('X_train:', X_train_b.shape, 'X_test:', X_test_b.shape)

```

Listing .23: Preprocessing lanjutan Bengaluru.



Gambar 19: Hasil train-test split dan scaling dataset Bengaluru.

Alasan penggunaan masing-masing teknik:

- **One-hot encoding** digunakan untuk `area_type` dan `availability` karena keduanya adalah variabel kategorikal nominal dengan jumlah kategori yang sedikit, sehingga one-hot encoding tidak menghasilkan terlalu banyak kolom baru dan mencegah model menganggap adanya hubungan ordinal.
- **Label encoding** digunakan untuk `location` karena kolom ini memiliki ratusan kategori unik. Jika menggunakan one-hot encoding, jumlah kolom akan meledak dan menyebabkan *curse of dimensionality*. Label encoding lebih efisien untuk kasus ini meskipun mengasumsikan adanya urutan.
- **Train-test split 80:20** digunakan agar model dapat dievaluasi secara objektif pada data yang belum pernah dilihat selama pelatihan.
- **Standardization** digunakan karena fitur-fitur seperti `bath`, `balcony`, dan `size_bhk` memiliki skala yang berbeda-beda. Standardization membuat semua fitur memiliki rata-rata 0 dan simpangan baku 1, sehingga algoritma ML yang sensitif terhadap skala (seperti regresi linear atau SVM) dapat bekerja dengan baik.

Kesimpulan

Berdasarkan praktikum yang telah saya lakukan, saya memahami beberapa hal berikut:

1. EDA digunakan untuk memahami struktur, distribusi, kualitas, dan hubungan antar-atribut sebelum pemodelan.
2. Data preprocessing diperlukan untuk menangani missing value dan duplikasi serta mengubah data ke format yang sesuai bagi algoritma ML.
3. Encoding mengubah atribut kategorikal menjadi representasi numerik, sedangkan train-test split dan scaling membantu menyiapkan data untuk pemodelan secara lebih objektif.

Praktikum ini memberi saya dasar untuk mengolah dan menganalisis data memakai Python.

Daftar Pustaka

- [1] Dicoding Indonesia, “Machine Learning Workflow: Di Balik Model AI yang Sukses,” 23 Oktober 2024. [Online]. Tersedia: <https://www.dicoding.com/blog/machine-learning-workflow-langkah-langkah-praktis-untuk-membangun-model-ai/>.
- [2] IBM, “Apa Itu Analisis Data Eksplorasi?” [Online]. Tersedia: <https://www.ibm.com/id-id/think/topics/exploratory-data-analysis>.
- [3] Revou, “Panduan Data Cleaning dengan Python Pandas dan Contohnya.” [Online]. Tersedia: <https://www.revou.co/panduan-teknis/python-data-cleaning>.
- [4] Digital Skola, “Data Preprocessing: Definisi, Tahapan, dan Implementasinya.” [Online]. Tersedia: <https://digitalskola.com/blog/data-science/data-preprocessing>.
- [5] Google for Developers, “Working with categorical data.” [Online]. Tersedia: <https://developers.google.com/machine-learning/crash-course/categorical-data>.
- [6] The pandas Development Team, “pandas.DataFrame.describe.” [Online]. Tersedia: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.describe.html>.
- [7] Scikit-learn Developers, “Preprocessing data.” [Online]. Tersedia: <https://scikit-learn.org/stable/modules/preprocessing.html>.
- [8] Scikit-learn Developers, “Common pitfalls and recommended practices.” [Online]. Tersedia: https://scikit-learn.org/stable/common_pitfalls.html.